

Додаток Б

Вихідний код ключових модулів

Б.1. Серверна частина — app.py (Flask API)

```
"""
NetVigil – Вебдодаток з використанням LLM для аналізу мережевого трафіку.
Дипломна робота Курмаша Р.В., 41-K, 2026
"""

import os, json, time, struct, hashlib
from datetime import datetime
import httpx
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS

app = Flask(__name__, static_folder="static", static_url_path="")
CORS(app)

LOKI_URL = os.getenv("LOKI_URL", "http://loki:3100")
OLLAMA_URL = os.getenv("OLLAMA_URL", "http://ollama:11434")
PROMETHEUS_URL = os.getenv("PROMETHEUS_URL", "http://prometheus:9090")
LLM_MODEL = os.getenv("LLM_MODEL", "llama3")

def _loki_query(query, limit=100, start=None, end=None):
    params = {"query": query, "limit": limit, "direction": "backward"}
    if start: params["start"] = start
    if end: params["end"] = end
    resp = httpx.get(f"{LOKI_URL}/loki/api/v1/query_range",
    params=params, timeout=10.0)
    return resp.json().get("data", {}).get("result", []) if resp.status_code == 200 else []

def _ollama_generate(prompt, format_json=True):
    payload = {"model": LLM_MODEL, "prompt": prompt, "stream": False}
    if format_json: payload["format"] = "json"
    resp = httpx.post(f"{OLLAMA_URL}/api/generate", json=payload, timeout=120.0)
    if resp.status_code == 200:
        raw = resp.json().get("response", "")
        return json.loads(raw) if format_json else {"text": raw}
    return None
```

```
@app.route("/api/dashboard")
def api_dashboard():
    # Агреговані KPI-дані: events, errors, alerts, CPU
    ...

@app.route("/api/logs")
def api_logs():
    # Фільтрація логів: container, keyword, hours, limit
    ...

@app.route("/api/analyze", methods=["POST"])
def api_analyze():
    # Аналіз логів через LLM з класифікацією загроз
    ...

@app.route("/api/analyze/pcap", methods=["POST"])
def api_analyze_pcap():
    # Завантаження, парсинг та AI-аналіз PCAP-файлів
    ...

@app.route("/api/chat", methods=["POST"])
def api_chat():
    # Чат-інтерфейс з LLM-асистентом
    ...
```

Повний вихідний код доступний у файлі `app-infrastructure/web-app/app.py`

Б.2. AI-Adapter — adapter.py (фоновий аналізатор)

```
"""
AI Adapter – фоновий мікросервіс для автоматичного аналізу логів
"""

async def fetch_logs(client, url, query, start_ns, end_ns):
    """Отримання логів з Loki за LogQL-запитом"""
    params = {"query": query, "start": str(int(start_ns)),
              "end": str(int(end_ns)), "limit": 1000}
    response = await client.get(url, params=params, timeout=10.0)
    ...

async def analyze_logs(client, url, model, logs_text):
    """Відправка логів до Ollama для AI-аналізу"""
    prompt = f"""Analyze these system logs for security anomalies...
```

```

Return JSON with risk_score, threat_found, problem_type..."
response = await client.post(f"{url}/api/generate",
json=payload, timeout=60.0)
...
"""
async def send_alert(client, url, analysis):
    """Генерація алерту в Alertmanager при risk_score >= 7"""
    payload = [{
        "labels": {"alertname": "SecOpsAIAnomalyDetected",
        "severity": severity, "risk_score": str(risk_score)},
        "annotations": {"summary": f"SecOps AI: {problem_type}",
        "description": analysis.get("description")}]
    }]
    ...
    """
async def main():
    """Основний цикл: сканування кожні 60 секунд"""
    while True:
        await asyncio.sleep(SCAN_INTERVAL)
        logs = await fetch_logs(...)
        if logs:
            analysis = await analyze_logs(...)
            if analysis and analysis.get("risk_score", 0) >= 7:
                await send_alert(...)

```

Повний вихідний код доступний у файлі `app-infrastructure/ai-adapter/adapter.py`

Б.3. Telegram Bot — bot.py (сповіщення)

```

"""
Telegram Bot – сервіс сповіщень з LLM-збагаченням алертів
"""
"""
@app.post("/alert")
async def handle_alert(request: Request):
    """Обробка алертів від Alertmanager"""
    for alert in alerts:
        if alert_name == "SecOpsAIAnomalyDetected":
            # Алерт від AI-адаптера – з повним описом
            message = format_ai_alert(alert)
        else:
            # Стандартний інфраструктурний алерт
            logs = await get_loki_logs(container)

```

```
ai_analysis = await analyze_with_llm(container, logs)
message = format_enriched_alert(alert, ai_analysis)
await send_to_telegram(message)
```

Повний вихідний код доступний у файлі `app-infrastructure/telegram-bot/bot.py`

Б.4. Docker Compose — `docker-compose.monitoring.yml` (фрагмент)

```
web-app:
  build:
    context: ./web-app
    container_name: web-app
  ports:
    - "5000:5000"
  environment:
    - LOKI_URL=http://loki:3100
    - OLLAMA_URL=http://ollama:11434
    - PROMETHEUS_URL=http://prometheus:9090
    - ALERTMANAGER_URL=http://alertmanager:9093
    - LLM_MODEL=llama3
  networks:
    - monitor-net
    - apps-net
  depends_on:
    - ollama
    - alertmanager
  restart: unless-stopped
```

Б.5. Jenkinsfile (CI/CD Pipeline)

```
pipeline {
  agent any
  stages {
    stage('1. Підготовка') {
      steps { cleanWs(); checkout scm; /* credentials, networks */ }
    }
    stage('3. Full Deploy') {
      steps {
        sh """
        cd app-infrastructure
        docker compose -f docker-compose.apps.yml up -d
        docker compose -f docker-compose.monitoring.yml build --no-cache
```

```
docker compose -f docker-compose.monitoring.yml up -d
"""
}
}
stage('5. Health Checks') {
  steps {
    script {
      def containers = ['nginx-proxy', 'mysql-db', 'prometheus',
        'telegram-bot', 'ai-adapter', 'web-app']
      for (c in containers) {
        sh "docker ps -f name=^/${c}$ -f status=running --quiet | grep ."
      }
    }
  }
}
}
```